✓26/32

SCHOOL OF COMPUTER SCIENCE

TAYLOR'S UNIVERSITY

ITS61504 DATA MINING

GROUP ASSIGNMENT 3: STROKE DETECTION

Instructions to students:

- The assignment should be attempted in groups.
- Complete this cover sheet and attach it to your submission – this should be your first page.

Module Learning Outcome:

- MLO3 - Work collaboratively to design, execute, and complete a predictive modelling project, demonstrating accountability, autonomy, and ethical responsibility

Student declaration:

We declare that:

- ***We understand what is meant by plagiarism.***
- ***The implication of plagiarism and usage of AI generative tool have been explained to us by our lecturer.***

This project is all our work, and we have acknowledged any use of the published or unpublished works of other people.

Name	Student ID	Section Number	Program
Angelica Julie Herliman	0376752	5	BCS
Arissa Sofia Binti Azlan Shah	0368867	5	BCS
Chia Shuen	0365866	5	BCS
Clarissa Clara	0377644	5	BCS
Zahra Sohrabirad	0375353	5	BCS

Avoid copy and paste job in your report and it is considered plagiarism. Plagiarism in all forms is forbidden. Students who submit plagiarized assignments will deserve 0 marks.

Table of Contents

1. Work Breakdown Structure (WBS) 3
2. Background and Project/Business Goals 4
3. Description of Dataset 5
4. Data-Related Issues and Corresponding Pre-Processing 6
5. Pre-Processing Activities 8
    5.1 Import Libraries 8
    5.2 Data Collection 8
    5.3 Identify and Handle Missing Values 8
    5.4 Dropping Unnecessary Columns 9
    5.5 Normalize Categorical Strings 10
    5.6 Remove Unique Values 10
    5.7 Remove Outliers 10
    5.8 Encoding 11
    5.9 Feature Scaling 11
6. Data Mining Techniques 12
    6.1 Logical Regression 12
    6.2 Decision Tree 14
    6.4 Support Vector Machine 16
7. Validation Techniques 18
    7.1 Logical Regression 18
    7.2 Decision Tree 19
    7.4 Support Vector Machine 21
8. Conclusion and Future Improvements 22
9. References and Appendix 23

1. Work Breakdown Structure (WBS)

This project was executed by a team of five members, with tasks clearly distributed to ensure timely completion and focus on specialized areas.

Member	**Primary Responsibilities (40%)**	**Secondary Responsibilities (60%)**
Angelica Julie Herliman	Data Acquisition, Data Set Description (Section 3), Identifying Data Issues (Section 4), Data Pre-processing (Section 5), Report Formatting and APA References (Section 10).	Initial code drafting and debugging.
Arissa Sofia Binti Azlan Shah	Data Mining Technique Selection and Justification (Section 6), Model Training, Validation Strategy (Section 7), Conclusion (Section 8), Report Formatting and APA References (Section 10).	Final Python Code implementation (Section 9) and results interpretation.
Chia Shuen	Data Mining Technique Selection and Justification (Section 6), Model Training, Validation Strategy (Section 7), Conclusion (Section 8), Report Formatting and APA References (Section 10).	Final Python Code implementation (Section 9) and results interpretation.
Clarissa Clara	Data Mining Technique Selection and Justification (Section 6), Model Training, Validation Strategy (Section 7), WBS creation (Section 1), Conclusion (Section 8), Report Formatting and APA References (Section 10).	Final Python Code implementation (Section 9) and results interpretation.
Zahra Sohrabirad	Background and Project Goal (Section 2), WBS creation (Section 1), Conclusion (Section 8), Report Formatting and APA References (Section 10).	Synthesis of results and final report review.



2. Background and Project/Business Goals

Stroke is one of the most important challenges in global health and is between the leading reason for mortality and long-term disability. If detected early and intervened at the right time, the severity of stroke outcomes can significantly reduce, yet traditional diagnostic approaches often rely on clinical symptoms that only appear after significant neurological damage has occurred. With the rapid advancement of data-driven techniques, machine learning (ML) has emerged as a powerful tool for predicting and detecting stroke risk based on patient characteristics before clinical onset. This study uses the stroke prediction dataset available on Kaggle (Fedesoriano, 2020), which has features related to demographic factors, medical history, lifestyle behaviors, and physiological measurements, such as age, hypertension, heart disease, BMI, glucose levels, and smoking status.

The main goal of this research is to develop an effective predictive framework that can identify individuals at risk of stroke using supervised machine learning techniques. To ensure analytical reliability, an extensive preprocessing pipeline was implemented, including removing irrelevant features, normalizing classified strings, handling rare values, dealing with outliers, encoding classified features, and feature scaling. These steps were necessary to convert the raw dataset into a format suitable for robust model development.

Three main classification algorithms were applied and compared. logistic regression, decision tree and support vector machine (SVM). Each algorithm has distinct advantages. logistic regression provides interpretability and probabilistic outputs, decision tree models provide transparent rule-based classification, and SVM can handle complex decision boundaries through optimal hyperplanes. Model performance was evaluated using confusion matrices, accuracy scores, and classification reports to assess predictive capabilities under class imbalance conditions, where stroke cases comprised less than 5% of the dataset.

The project does not only aim to determine the most effective model for stroke prediction but also demonstrates how machine learning can support medical decisions by identifying risk early, helping doctors prioritize high-risk individuals and improving preventive care strategies. These findings contribute to ongoing efforts in healthcare analytics and underscore the value of predictive modeling in improving patient outcomes and advancing data-driven medical practices.

3. Description of Dataset

The dataset used is the Stroke Prediction Dataset created by Fedesoriano in 2020.

- Source: Kaggle
- Observations: 5110 instances (rows)
- Features (inputs): 4 quantitative, continuous variables:
    - Identification Number
    - Age (years)
    - Average Glucose Level (mg/dL)
    - Body Mass Index

: 8 qualitative, categorical variables:

    - Gender (male or female)
    - Hypertension (0 = no, 1 = yes)
    - Heart Disease (0 = no, 1 = yes)
    - Ever Married (no or yes)
    - Work Type ("children", "Govt_jov", "Never_worked", "Private" or "Self-employed")
    - Residence Type ("Rural" or "Urban")
    - Smoking Status ("formerly smoked", "never smoked", "smokes" or "Unknown")
- Target (output): 1 categorical variable:
    - Stroke (0 = no, 1 = yes)

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1
...	...	...	...	...	...	...	...	...	...	...	...	...
5105	18234	Female	80.0	1	0	Yes	Private	Urban	83.75	NaN	never smoked	0
5106	44873	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0
5107	19723	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.6	never smoked	0
5108	37544	Male	51.0	0	0	Yes	Private	Rural	166.29	25.6	formerly smoked	0
5109	44679	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.2	Unknown	0

5110 rows × 12 columns

Figure 1 Raw Dataset

Summary statistics show age ranges from 0.08 to 82 years, BMI ranges from 10.3 to 97.6, and glucose levels range from 55.12 to 271.74. Binary medical indicators such as hypertension and heart disease take values of 0 or 1. Several categorical fields also contain rare categories such as 'other' in gender and 'Unknown' in smoking_status. 3/4

4. Data-Related Issues and Corresponding Pre-Processing

The dataset used is still in the form of unclean data, so data cleaning and data pre-processing must be carried out so that this data can become clean data that can be used by the model to perform analysis that produces accurate predictions. This dataset contains several numerical variables such as age, bmi, and average glucose level, along with several categorical attributes such as gender, ever_married, work_type, residence_type, and smoking_status. There is also an identifier column, id, which serves only as a unique identifier for each record.

Issue Identified	Description	Corresponding Pre-Processing activity
Missing Values	The dataset has missing values in the BMI column where several records contain empty or NaN values.	Imputation or removal depending on severity.
Irrelevant / Unnecessary Column	The id column is not needed because it is only an identifier.	Drop the id column

Inconsistent Categorical Formatting	There are inconsistencies in capital letters and spacing in categorical attributes such as gender, ever_married, work_type, residence_type, and smoking_status.	Standardize string formatting (lowercase, trimming whitespace).
Rare Categories / Unstable Classes	Several categorical values rarely appear such as “other” in gender or “unknown” in work_type, which may lead to unstable model representation.	Combine rare categories or remove them.
Out-of-Range / Invalid Values	The dataset contains unrealistic values such as BMI below 10 or above 60, average glucose level around 600, and ages below 0 or above 120.	Handle by removal, capping, or correction.
String Values Not Compatible with ML Models	Machine learning models do not accept string formats.	Label encoding: gender, ever_married One-hot encoding: residence_type, work_type, smoking_status
Feature Scaling Required	Numerical variables have very different ranges (age 0–120, BMI 10–60, glucose 0–600). Without scaling, models become sensitive to magnitude.	Apply normalization or standardization.

5/6

5. Pre-Processing Activities

5.1 Import Libraries

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
```

Figure 2 Importing Necessary Libraries

5.2 Data Collection

```
stroke_df = pd.read_csv('stroke_dataset.csv')
stroke_df
```

Figure 3 Importing Dataset Using the Read Function

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1
...	...	...	...	...	...	...	...	...	...	...	...	...
5105	18234	Female	80.0	1	0	Yes	Private	Urban	83.75	NaN	never smoked	0
5106	44873	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0
5107	19723	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.6	never smoked	0
5108	37544	Male	51.0	0	0	Yes	Private	Rural	166.29	25.6	formerly smoked	0
5109	44679	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.2	Unknown	0

5110 rows × 12 columns

Figure 4 Raw Dataset

5.3 Identify and Handle Missing Values ✓

```
stroke_df.isnull()
```

Figure 5 Identifying Missing Values

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	True	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...
5105	False	False	False	False	False	False	False	False	False	True	False	False
5106	False	False	False	False	False	False	False	False	False	False	False	False
5107	False	False	False	False	False	False	False	False	False	False	False	False
5108	False	False	False	False	False	False	False	False	False	False	False	False
5109	False	False	False	False	False	False	False	False	False	False	False	False

5110 rows × 12 columns

Figure 6 Output of Missing Values

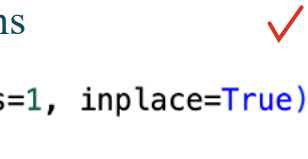
```
stroke_df['bmi'].fillna(stroke_df['bmi'].median(), inplace=True)
```

Figure 7 Handle Missing Values Using fillna Function, Imputing Median Value on Missing Values

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	28.1	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Figure 8 Output after Handling of Missing Values

5.4 Dropping Unnecessary Columns



```
stroke_df.drop('id', axis=1, inplace=True)
stroke_df.head()
```

Figure 9 Dropping the 'id' Column

	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	28.1	never smoked	1
2	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Figure 10 Output After Dropping the 'id' Column

The id column has no contribution to modeling so the id column in the dataset is removed using the '.drop()' function.



5.5 Normalize Categorical Strings

```
cat_cols = ['gender','ever_married','work_type','Residence_type','smoking_status']
for c in cat_cols:
    if c in stroke_df.columns:
        stroke_df[c] = stroke_df[c].astype(str).str.strip().str.lower()
```

Figure 11 Normalization of Categorical Strings

The str.lower() function changes the entire categorical values to lowercase letters. The str.strip() function is carried out to remove leading and trailing characters from a string and remove whitespace characters to normalize and ease data mining process.

5.6 Remove Unique Values

```
if 'gender' in stroke_df.columns and (stroke_df['gender'] == 'other').sum() > 0:
    print("gender 'other' count:", (stroke_df['gender']=='other').sum())
    stroke_df = stroke_df[stroke_df['gender'] != 'other']

allowed_work = ['private','self-employed','children','govt_job','never_worked']
if 'work_type' in stroke_df.columns:
    stroke_df.loc[~stroke_df['work_type'].isin(allowed_work), 'work_type'] = 'unknown'

allowed_res = ['urban','rural']
if 'residence_type' in stroke_df.columns:
    stroke_df.loc[~stroke_df['residence_type'].isin(allowed_res), 'residence_type'] = 'unknown'

allowed_smoke = ['formerly smoked','never smoked','smokes','unknown']
if 'smoking_status' in stroke_df.columns:
    stroke_df.loc[~stroke_df['smoking_status'].isin(allowed_smoke), 'smoking_status'] = 'unknown'
```

Figure 12 Remove Unique Values

Categories that rarely appear or are outside the category are changed to the form “unknown”.

5.7 Remove Outliers

```
if 'age' in stroke_df.columns:
    stroke_df = stroke_df[(stroke_df['age'] > 0) & (stroke_df['age'] <= 120)]
if 'bmi' in stroke_df.columns:
    stroke_df = stroke_df[(stroke_df['bmi'] >= 10) & (stroke_df['bmi'] <= 60)]
if 'avg_glucose_level' in stroke_df.columns:
    stroke_df = stroke_df[(stroke_df['avg_glucose_level'] > 0) & (stroke_df['avg_glucose_level'] <= 600)]
```

Figure 13 Remove Outliers

Data with extreme and unreasonable values will be removed so that only rows with realistic values remain.

5.8 Encoding

```
label_encoder = LabelEncoder()
if 'gender' in stroke_df.columns:
    stroke_df['gender'] = label_encoder.fit_transform(stroke_df['gender'])
if 'ever_married' in stroke_df.columns:
    stroke_df['ever_married'] = label_encoder.fit_transform(stroke_df['ever_married'])

stroke_df = pd.get_dummies(stroke_df, columns=[c for c in ['Residence_type','work_type','smoking_status']
                                               if c in stroke_df.columns], drop_first=True, dtype='int')
```

Figure 14 Encoding

The machine learning model does not accept strings, so the data is converted into a numeric variable. Label encoding and One-hot encoding are used, ensuring that each category is represented consistently without changing the meaning of the data.

5.9 Feature Scaling

```
scaler = StandardScaler()
stroke_df[['age']] = scaler.fit_transform(stroke_df[['age']])

min_max_scaler = MinMaxScaler()
stroke_df[['avg_glucose_level']] = min_max_scaler.fit_transform(stroke_df[['avg_glucose_level']])
stroke_df[['bmi']] = min_max_scaler.fit_transform(stroke_df[['bmi']])
```

Figure 15 Feature Scaling

Because the ranges of the variables differ significantly, scaling is used to adjust the scale of the numeric features so that all variables are within comparable ranges. This prevents features with large values from dominating the model and helps the algorithm perform more stably and accurately.

	gender	age	hypertension	heart_disease	ever_married	avg_glucose_level	bmi	stroke	Residence_type_urban	work_type_govt_job	work_type_never_worked	work_type_private	wo
0	1	1.050324	0	1	1	0.801265	0.532389	1	1	0	0	1	
1	0	0.785174	0	0	1	0.679023	0.360324	1	0	0	0	0	
2	1	1.624816	0	1	1	0.234512	0.449393	1	0	0	0	1	
3	0	0.254874	0	0	1	0.536008	0.487854	1	1	0	0	1	
4	0	1.580625	1	0	1	0.549349	0.277328	1	0	0	0	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	
5105	0	1.624816	1	0	1	0.132167	0.360324	0	1	0	0	1	
5106	0	1.669008	0	0	1	0.323516	0.601215	0	1	0	0	0	
5107	0	-0.363810	0	0	1	0.128658	0.410931	0	0	0	0	0	
5108	1	0.343257	0	0	1	0.513203	0.309717	0	0	0	0	1	
5109	0	0.033916	0	0	1	0.139230	0.321862	0	1	1	0	0	

5096 rows × 16 columns

Figure 16 Final Clean Dataset

6. Data Mining Techniques

6.1 Logical Regression

Technique: Logical Regression Classification

Logical regression is a statistical prediction model that relates predictor variables to a binary response using the sigmoid, or logistic, function. This function maps input values to probabilities between 0 and 1, allowing for binary classification. The model is typically used for simple two-way or, at most, three-way interactions. Although it can model complex nonlinear relationships, this requires substantial transformation of the predictor variables (Tu, J. V., 1996).

To achieve optimal accuracy, input variables often need to be encoded in binary form (Ruczinski et al., 2012). However, when important interactions are not explicitly modelled, logistic regression may produce less accurate results compared to more advanced algorithms such as neural networks.

Justification:

Logistic regression is widely used for classification analysis, particularly for predicting categorical outcomes. This machine learning algorithm was chosen based on the dataset and the problem we aimed to solve, classifying whether the input variables would ultimately lead to a stroke or not (Frost, J., n.d.). Since the dependent variable is binary (stroke or no stroke), logistic regression is an appropriate and effective choice.

Logistic regression also offers clear insights into feature importance, making the extracted information easier to interpret. Because the algorithm performs well on small to medium-sized datasets, our dataset of 5,000 rows is suitable and not large enough to overwhelm the model. Additionally, logistic regression provides probabilistic outputs, which can help doctors make more informed decisions based on predicted likelihoods. Finally, logistic regression performs best when the independent variables are not highly correlated, a condition that our dataset satisfies (Song, P., 2025).

```
df_dummies = pd.get_dummies(stroke_df, drop_first=True)
# Shuffle Rows Prior to Splitting Data into Features (X) and Outcome (Y)
from sklearn.utils import shuffle
from sklearn.model_selection import train_test_split

df_shuffled = shuffle (df_dummies, random_state=42)

DV = 'stroke' # DV => Dependent variable

x = df_shuffled.drop (DV, axis=1) # axis=1, drop the DV column, don't axis = 0
y = df_shuffled [DV] # this our depenedent variable?

# 20% for training, The rest for testing
# random_state = 42 is used to select the 20% of the data set for testing random
x_train, x_test, y_train, y_test = train_test_split (x, y, test_size=0.20, randc

print (x_train.head ())
```

Figure 17 Train Test Splitting

Splitting 80 percent of the dataset for training and learning, 20 percent of the dataset for testing and validation to ensure the dataset is not used to the training dataset and avoid overfitting and underfitting.

```
      gender       age  hypertension  heart_disease  ever_married  \
3539       1 -1.424410             0              0             0
3483       0 -1.336027             0              0             0
4875       0 -0.098659             0              0             1
2881       0  0.829366             0              0             1
4905       0  0.740983             0              0             1

      avg_glucose_level       bmi  Residence_type_urban  work_type_govt_job  \
3539           0.062321  0.159919                     0                   0
3483           0.064122  0.265182                     0                   0
4875           0.173207  0.678138                     0                   0
2881           0.169421  0.188259                     1                   0
4905           0.135814  0.265182                     0                   0

      work_type_never_worked  work_type_private  work_type_self-employed  \
3539                       0                 0                        0
3483                       0                 0                        0
4875                       0                 1                        0
2881                       0                 0                        1
4905                       0                 1                        0

      smoking_status_never smoked  smoking_status_smokes  \
3539                            0                      0
3483                            0                      0
4875                            1                      0
2881                            0                      0
4905                            0                      1

      smoking_status_unknown
3539                       1
3483                       1
4875                       0
2881                       1
```

Figure 18 Training Dataset

```
[12]
✓ 0s
from sklearn.linear_model import LogisticRegression
LRmodel=LogisticRegression()

[13]
✓ 0s
LRmodel.fit(x_train, y_train)

▾ LogisticRegression
LogisticRegression()

[14]
✓ 0s
LR_pred=LRmodel.predict(x_test)
```

Figure 19 Import Logistic Regression Model, Fitting in the Training Dataset before Testing

6.2 Decision Tree

Technique: Decision Tree Classification

Decision Tree is a supervised machine learning technique for classification and regression applications. It operates by dividing the dataset into subgroups according to feature values. Every split is predicated on a condition that best predicts the goal for regression or divides the classes for classification. The tree is made up of root nodes, internal nodes, and leaf nodes.

- Root node: the initial point of decision and the characteristic that divides the data most effectively.
- Internal node: Features that further divide data.
- Leaf nodes: the end result and predicted value.

Justification:

Decision tree structure can be visualized and understood easily. Because the rules can be read using “if... then...” statement. This technique can split on any type of feature without special preprocessing, and it only uses features that contribute to splitting, lessening the need for manual feature selection. It is flexible and widely applicable because it can be used for classification and regression.

```
X = stroke_df.drop('stroke', axis=1)
y = stroke_df['stroke']
```

Figure 20 Separate features and target

This code separates the dataset into features, x and target variable, y. x has all the input variables, while y has the stroke label - 0 represents no stroke, while 1 represents stroke.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Figure 21 Train test splitting

This code separates data into 80% for training and 20% for testing set.

```
dt_clf = DecisionTreeClassifier(random_state=42, max_depth=5)
dt_clf.fit(X_train, y_train)
```

DecisionTreeClassifier

DecisionTreeClassifier(max_depth=5, random_state=42)

Figure 22 Build and train Decision Tree

```
y_pred = dt_clf.predict(X_test)
```

Figure 23 Predict on test data

6.4 Support Vector Machine

Technique: Support Vector Machine

Support Vector Machine (SVM) is a supervised machine learning that is mostly utilized for classification tasks. It works by identifying the best hyperplane, which also is called a boundary, that divides the various classes in the data. SVM selects the line that maximizes the margin or the separation between the boundary and the nearest data points from each class, known as support vectors, rather than just drawing any line. SVM enhances the model's capacity to generalize new and unseen data. SVM can also handle both linear and non-linear problems by utilizing different kernels such as linear, polynomial, or RBF. The effectiveness of this method for pattern recognition and classification has been widely documented in machine learning literature including Bishop (2006) and formally introduced by Cortes and Vapnik (1995).

Justification:

Using Support Vector Machine (SVM) to predict stroke because it is highly effective on datasets that may be high-dimensional, imbalanced, or contain complex decision boundaries which are all common features in medical prediction tasks. SVM is also effective for binary classification problems. The capacity of SVM to maximize the margin between classes helps in improving the generalization and lowering the misclassification of key instances. Stroke prediction often depends on a variety of clinical parameters including gender, age, work type, and smoking status factors.

```
X = stroke_df.drop('stroke', axis=1)  # Features
y = stroke_df['stroke']  # Target variable
```

Figure 24 Separate the Features and the Target

This part of the code separates the features and the target. X is equal to all the columns except the target then as for Y, is the only target column. This prepares data for machine learning.

```
# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Figure 25 Split the Dataset into Training and Testing

Then the data is split into training and testing sets which 80% is for training and 20% for testing. The random state is used to ensure reliability.

```
# Initialize the SVM classifier
svm_classifier = SVC(kernel='linear')
```

Figure 26 Initialize the SVM model

Then we initialize an SVM model from scikit-learn, and we use kernel=’linear’ to separate the boundary faster.

```
# Train the SVM classifier on the training data
svm_classifier.fit(X_train, y_train)
```

```
SVC(kernel='linear')
```

Figure 27 Train the Classifier

The model learns the patterns from the training dataset, so it calculates the hyperplane, the support vectors, and the max margin.

```
# Predict the classes for the test set
y_pred = svm_classifier.predict(X_test)
```

Figure 28 Make the Predections

Finally, the code shows the predictions of the test set. The train SVM uses the learned boundary to predict whether a person has a stroke or not by using the y_pred which stores those predictions. This research has demonstrated that SM is resistant to overfitting and works well when there are more features than samples, which has advantages in many healthcare datasets. (Cortes & Vapnik, 1995).

5/6

7. Validation Techniques

7.1 Logical Regression

```
[17]
✓ 0s
from sklearn.metrics import confusion_matrix
confusion_matrix = confusion_matrix(y_test, LR_pred)
print(confusion_matrix)

[[973   0]
 [ 47   0]]
```

Figure 29 Confusion Matrix

The confusion matrix shows 973 true negatives, which means it correctly predicted no stroke for people who did not have a stroke. 46 false negatives, which incorrectly predicted no stroke for people who did have a stroke. 0 false positives and 0 true positives show that the algorithm did not create any false alarms but never predicted a stroke at all when a stroke happened to someone.

```
[15]
✓ 0s
from sklearn.metrics import accuracy_score
accuracy_score(y_test, LR_pred)

0.953921568627451
```

Figure 30 Confusion Matrix, Accuracy Score

The accuracy score shows a whopping 95 percent accuracy, which means this model is suitable for the dataset.

```
Confusion matrix:
            Predicted No  Predicted Yes  Total
Actual No            973              0    973
Actual Yes            47              0     47
Total               1020              0   1020

Classification report:
              precision    recall  f1-score   support

           0       0.95      1.00      0.98       973
           1       0.00      0.00      0.00        47

    accuracy                           0.95      1020
   macro avg       0.48      0.50      0.49      1020
weighted avg       0.91      0.95      0.93      1020
```

Figure 31 Confusion Matrix Results, and Classification Report

7.2 Decision Tree

Accuracy score:

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9480392156862745

Confusion Matrix:
 [[967   3]
 [ 50   0]]

Classification Report:
              precision    recall  f1-score   support

           0       0.95      1.00      0.97       970
           1       0.00      0.00      0.00        50

    accuracy                           0.95      1020
   macro avg       0.48      0.50      0.49      1020
weighted avg       0.90      0.95      0.93      1020
```

Figure 32 Evaluate model

The accuracy score is predicted to be 94.8%. This accuracy does not fully represent performance because the dataset is imbalanced, and stroke cases are very few.

Confusion matrix:

```
from sklearn.metrics import confusion_matrix
import pandas as pd

cm_table = pd.DataFrame(
    cm,
    index=["Actual: No Stroke (0)", "Actual: Stroke (1)"],
    columns=["Predicted: No Stroke (0)", "Predicted: Stroke (1)"]
)

print(cm_table)
```

```
                       Predicted: No Stroke (0)  Predicted: Stroke (1)
Actual: No Stroke (0)                       967                      3
Actual: Stroke (1)                           50                      0
```

Figure 33 Confusion matrix table

True Negatives: The model correctly identified 967 people as not having a stroke.

False Positives: The model incorrectly predicted stroke when the person did not have stroke.

False Negatives: The model incorrectly detects 50 real stroke cases, so it heavily misses stroke patients.

True Positive: The model did not detect any stroke cases.

The model tends to predict no stroke for almost every patient because the dataset has only almost 5% cases are strokes.

Classification report:

For class 0, it has a precision value of 0.95, which means very few cases are mis predicted as no stroke, recall value of 1.00 means it correctly catches all no stroke cases and F1-score of 0.97 defines very strong performance.

For class 1, precision value of 0.00 means the algorithms always mis predicts stroke occurrence, recall value of 0.00 means never correctly detects actual stroke patients and F1-score of 0.00 means this model failed completely.

```
plt.figure(figsize=(25, 15))
plot_tree(
    dt_clf,
    feature_names=X.columns,
    class_names=['No Stroke', 'Stroke'],
    filled=True,
    fontsize=8,
    rounded=True
)
plt.show()
```

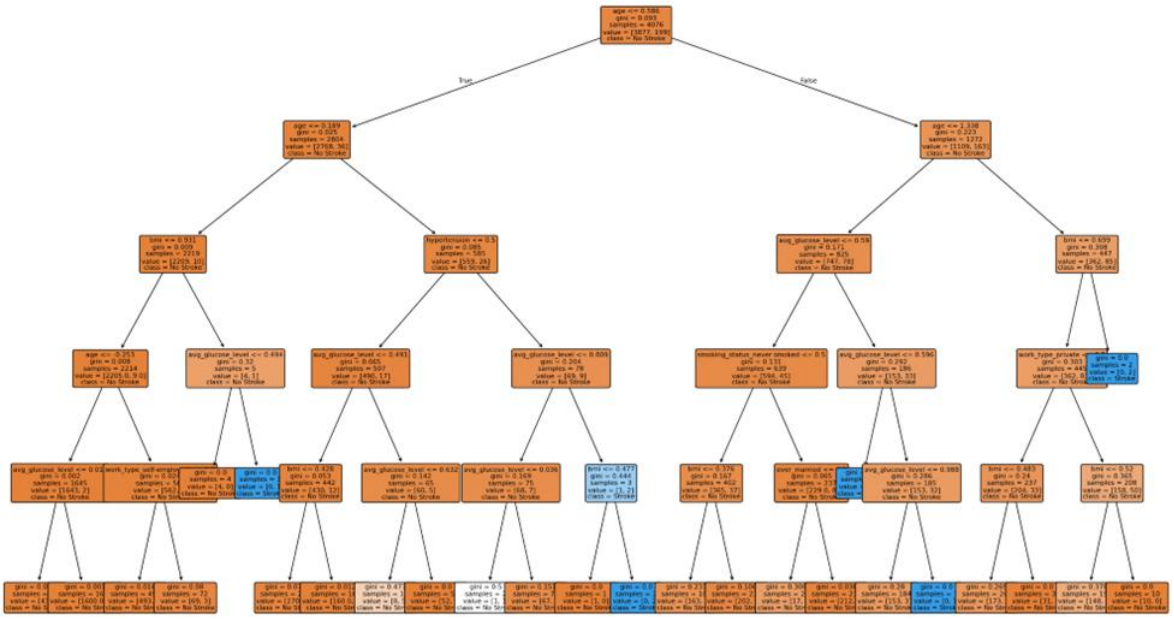


Figure 34 Classification Report

According to figure 13, most leaves show no stroke (orange boxes), and almost no leaves show stroke (blue boxes). The white box represents high impurity and no strong prediction. The light blue box represents patients that have strokes but with lower confidence and the light orange boxes represent patients that have no stroke but less confidence.

7.4 Support Vector Machine

```
# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

Accuracy: 0.9392156862745098
```



Figure 35 Accuracy Report

The model shows 93% of the data that shows ‘no stroke’, which is less than Decision Tree and Logical Regression.

```
from sklearn.metrics import accuracy_score, confusion_matrix

# Accuracy score
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)

Accuracy: 0.9392156862745098
Confusion Matrix:
 [[958   0]
 [ 62   0]]
```

Figure 36 Confusion Matrix Report

Shows that 958 are true negatives, which means that the model correctly predicted no stroke and 62 False Negatives that predicted no stroke for 62 actual stroke cases while false positives and true positives didn’t detect a single case of someone with or without stroke.



3/4

8. Conclusion and Future Improvements

This project used machine learning techniques to predict stroke risk by using the Kaggle Stroke Prediction Dataset. After executing a complete preprocessing pipeline, like handling missing values, normalizing categorical fields, addressing outliers, encoding variables, and scaling numerical features, the final dataset has been used to train three classification models: Logistic Regression, Decision Tree, and Support Vector Machine. Between these models, Logistic Regression achieved the highest accuracy at approximately 95%, showing strong and great performance in classifying non-stroke cases. Decision Tree and SVM produced a bit lower accuracy scores but still demonstrated consistent predictive ability. With these results, all models faced a significant challenge. The dataset is highly imbalanced, with stroke cases representing a very small part of the total observations. This led to limited recall for the positive class, indicating that the models frequently missed true stroke cases even when overall accuracy appeared high.

These findings highlight an important insight into medical prediction tasks. Accuracy alone is not enough as a measure of performance, especially when predicting rare but critical health events. The project achieved an important goal of comparing classification techniques and demonstrating how machine learning can assist in early risk assessment. With additional Reforms, such predictive systems can become valuable tools to support healthcare professionals in identifying high-risk individuals before severe outcomes occur.

For future improvements we can add the following:

1. Handle Class Imbalance: Apply SMOTE, oversampling, or class-weight adjustments to improve the detection of true stroke cases.
2. Try Advanced Models: Use Random Forest, XGBoost, or Neural Networks, which often perform better with imbalanced medical datasets.
3. Hyperparameter Tuning: Implement Grid Search or Random Search to optimize model parameters for better performance.
4. Feature Enhancements: Engineer additional clinical or derived features to strengthen predictive power.

2/2

9. References and Appendix

Bishop, C. M. 2007. Pattern recognition and machine learning. *Journal of Electronic Imaging*, *16*(4), 049901. https://doi.org/10.1117/1.2819119

Cortes, C., & Vapnik, V. 1995. Support-vector networks. *Machine Learning*, *20*(3), 273–297. https://doi.org/10.1007/bf00994018

Fedesoriano. 2020. Stroke Prediction Dataset. Kaggle. https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

Frost, J. Logistic Regression Overview with Example. Statistics By Jim. https://statisticsbyjim.com/regression/logistic-regression/

Ruczinski, I., Kooperberg, C., & LeBlanc, M. 2003. Logic Regression. *Journal of Computational and Graphical Statistics*, *12*(3), 475–511. https://doi.org/10.1198/1061860032238

Song, P. 2025, April 4. When to Use Logistic Regression: Comprehensive Guide. ML Journey. https://mljourney.com/when-to-use-logistic-regression-comprehensive-guide/

Tu, J. V. 1996. Advantages and disadvantages of using artificial neural networks versus logistic regression for predicting medical outcomes. *Journal of Clinical Epidemiology*, Volume 49, Issue 11, Pages 1225-1231, ISSN 0895-4356. https://doi.org/10.1016/S0895-4356(96)00002-9

X., C. 2022. Stroke Prediction Model (Binary Classification). Kaggle. https://www.kaggle.com/code/docxian/stroke-prediction

5/6

ASG3_Group2_Section5.pdf

Background and Project/Business Goal (4): 3
Data Set Description (4): 3
Data-Related Issues & Preprocessing (6): 5
Data Mining Techniques & Justification (6): 5
Model Validation (4): 3
Conclusion & Improvements (2): 2
Report Formatting (6): 5

Total (32%): 26

		Background and Project/Business Goal	Data Set Description	Data-Related Issues & Preprocessing	Data Mining Techniques & Justification	Model Validation	Conclusion & Improvements	Report Formatting	Total Report (32%)	Project Demo (Presentation) Total is 8%					Total (40%)
		4	4	6	6	4	2	6	32	Timing (5)	Delivery (5)	Understanding of Project (5)	Total (15)	Total (8%)	
61	Angelica Julie Herliman	3	3	5	5	3	2	5	26	3	4	4	11	5.87	31.87
62	Arissa Sofia Binti Azlan Shah	3	3	5	5	3	2	5	26	5	4	4	13	6.93	32.93
63	Chia Shuen	3	3	5	5	3	2	5	26	5	4	5	13	6.93	32.93
64	Clarissa Clara	3	3	5	5	3	2	5	26	5	4	4	13	6.93	32.93
65	Zahra Sohrabirad	3	3	5	5	3	2	5	26	5	4	4	13	6.93	32.93